

cho mình 1 vài câu về java core về tham trị tham chiếu, string, 1 vài câu dễ bị đánh lừa trong java. kèm lời giải thích vì sao lại như vậy. hãy nói thêm 1 chút về từ khoá final, this, super. vì sao phải tạo private constructor, vì sao phải final class, vì sao dùng abstract class và interface mà không dùng class luôn. mục đích của interface và abstract class, vì sao dùng, vì sao sinh ra 2 khái niệm đó, nên dùng cái nào, vì sao java không cho phép đa kế thừa, nói tất cả vấn đề liên quan đến 2 khái niệm đó và các vấn đề liên quan đến chúng

Mình sẽ gom theo kiểu phỏng vấn Java core: đưa câu hỏi bầy, giải thích bản chất JVM/biến/tham chiếu trước, rồi nối sang final, this, super, abstract class, interface và đa kế thừa.

Đã suy nghĩ trong 26s >

Dưới đây là một bộ Java Core rất hay bị hỏi trong phỏng vấn, đặc biệt là những câu dễ đánh lừa. Mình sẽ giải thích theo bản chất, không chỉ trả lời output.

1. Java là tham trị hay tham chiếu?

Câu trả lời chuẩn là:

Java luôn truyền tham trị — pass by value.

Nhưng chỗ dễ nhầm là: với object, Java truyền **bản copy của reference**, chứ không truyền bản thân object.

Ví dụ:

```
class User {  
    String name;  
  
    User(String name) {  
        this.name = name;  
    }  
}
```

```
public class Main {  
    static void changeName(User u) {  
        u.name = "B";  
    }  
  
    static void reassign(User u) {  
        u = new User("C");  
    }  
  
    public static void main(String[] args) {  
        User user = new User("A");  
  
        changeName(user);  
        System.out.println(user.name); // B  
  
        reassign(user);  
        System.out.println(user.name); // B, không phải C  
    }  
}
```

Rất nhiều người nhìn đoạn này rồi nói Java truyền tham chiếu. Nhưng thật ra không phải.

Biến `user` ở `main` đang giữ một địa chỉ tham chiếu tới object `User("A")`. Khi gọi `changeName(user)`, Java copy giá trị tham chiếu đó đưa vào biến `u`. Bây giờ `user` và `u` đều đang trỏ tới cùng một object. Vì vậy khi `u.name = "B"` thì object thật bị đổi, nên bên ngoài thấy đổi.

Nhưng ở `reassign(user)`, dòng này:

```
u = new User("C");
```

chỉ làm cho biến local `u` trỏ sang object mới. Biến `user` bên ngoài không bị đổi vì `u` chỉ là bản copy của reference.

Nói dễ hiểu: Java không đưa "cái nhà" cho method. Java đưa "bản photo địa chỉ nhà". Nếu method dùng địa chỉ đó để vào nhà sửa đồ, bên ngoài thấy thay đổi. Nhưng nếu method lấy tờ giấy địa chỉ đó gạch đi rồi viết địa chỉ nhà khác, tờ giấy gốc bên ngoài không đổi.

2. Câu bẫy kinh điển: swap object có được không?

```
class User {
    String name;

    User(String name) {
        this.name = name;
    }
}

public class Main {
    static void swap(User a, User b) {
        User temp = a;
        a = b;
        b = temp;
    }

    public static void main(String[] args) {
        User u1 = new User("A");
        User u2 = new User("B");

        swap(u1, u2);

        System.out.println(u1.name);
        System.out.println(u2.name);
    }
}
```

Output:

A
B

Không swap được.

Vì `a` và `b` chỉ là hai biến local nhận bản copy reference từ `u1`, `u2`. Khi đổi `a`, `b`, chỉ đổi trong method. `u1`, `u2` bên ngoài vẫn giữ reference cũ.

Muốn đổi dữ liệu thì phải đổi bên trong object:

```
static void swapName(User a, User b) {
    String temp = a.name;
    a.name = b.name;
    b.name = temp;
}
```

Lúc này đổi được vì đang mutate object thật.

3. String trong Java: vì sao rất dễ bị đánh lừa?

String trong Java là **immutable**, nghĩa là object String sau khi tạo ra thì không thay đổi được.

Ví dụ:

```
String s = "Java";  
s.concat(" Core");  
  
System.out.println(s);
```

Output:

```
Java
```

Vì `concat()` không sửa object "Java". Nó tạo ra String mới "Java Core", nhưng mình không gán lại nên mất reference.

Phải viết:

```
s = s.concat(" Core");  
System.out.println(s); // Java Core
```

4. Bẫy `==` và `equals()` với String

```
String s1 = "Java";  
String s2 = "Java";  
  
System.out.println(s1 == s2);  
System.out.println(s1.equals(s2));
```

Output:

```
true  
true
```

Vì "Java" là string literal. Java đưa nó vào **String Pool**. Hai biến `s1` , `s2` cùng trỏ tới một object trong pool.

Nhưng:

```
String s1 = "Java";  
String s2 = new String("Java");  
  
System.out.println(s1 == s2);  
System.out.println(s1.equals(s2));
```

Output:

```
false  
true
```

`==` so sánh reference, tức là có cùng object không.

`equals()` so sánh nội dung.

`new String("Java")` ép JVM tạo object mới ngoài heap, nên reference khác với object trong String Pool.

Trong dự án thật, khi so sánh nội dung String, gần như luôn dùng:

```
s1.equals(s2)
```

Nếu sợ null:

```
"Java".equals(s1)
```

Không nên viết:

```
s1 == s2
```

trừ khi thật sự muốn kiểm tra hai biến có trỏ cùng object hay không.

5. Bẫy compile-time String concat

```
String s1 = "Java";  
String s2 = "Ja" + "va";  
  
System.out.println(s1 == s2);
```

Output:

```
true
```

Vì "Ja" + "va" là hằng số compile-time. Compiler tự gộp thành "Java" trước khi chạy chương trình.

Nhưng:

```
String part = "Ja";  
String s1 = "Java";  
String s2 = part + "va";  
  
System.out.println(s1 == s2);
```

Output thường là:

```
false
```

Vì `part + "va"` xảy ra runtime, tạo object String mới.

Nếu muốn đưa vào pool:

```
String s2 = (part + "va").intern();  
System.out.println(s1 == s2); // true
```

Nhưng trong code thực tế, không nên lạm dụng `intern()` nếu không hiểu rõ memory behavior.

6. Bẫy String truyền vào method

```
public class Main {  
    static void change(String s) {  
        s = s + " Core";  
    }  
  
    public static void main(String[] args) {  
        String text = "Java";  
        change(text);  
        System.out.println(text);  
    }  
}
```

Output:

Java

Vì String immutable và Java pass-by-value. Biến `s` trong method nhận bản copy reference. Khi `s = s + " Core"`, nó trỏ sang object mới. Biến `text` bên ngoài vẫn trỏ tới "Java".

Muốn đổi thì method phải return:

```
static String change(String s) {  
    return s + " Core";  
}  
  
public static void main(String[] args) {  
    String text = "Java";  
    text = change(text);  
}
```

7. Bẫy StringBuilder và String

```
static void change(StringBuilder sb) {  
    sb.append(" Core");  
}  
  
public static void main(String[] args) {  
    StringBuilder sb = new StringBuilder("Java");  
    change(sb);  
    System.out.println(sb);  
}
```

Output:

Java Core

Vì `StringBuilder` mutable. Method nhận copy reference, nhưng cả hai reference cùng trỏ tới một object. Gọi `append()` là sửa object thật.

Nhưng:

```
static void change(StringBuilder sb) {  
    sb = new StringBuilder("Python");  
}  
  
public static void main(String[] args) {  
    StringBuilder sb = new StringBuilder("Java");  
    change(sb);  
    System.out.println(sb);  
}
```

Output:

Java

Vì mình chỉ reassign biến local `sb` trong method, không ảnh hưởng biến ngoài.

8. Bẫy `final` với object


```
final StringBuilder sb = new StringBuilder("Java");
sb.append(" Core");

System.out.println(sb);
```

Output:

```
Java Core
```

Không lỗi.

Vì `final` với biến object không có nghĩa object bất biến. Nó chỉ có nghĩa là biến đó không được trở sang object khác.

Cái này lỗi:

```
final StringBuilder sb = new StringBuilder("Java");
sb = new StringBuilder("Python"); // lỗi compile
```

Nhớ kỹ:

```
final reference != immutable object
```

Muốn object thật sự immutable thì class phải thiết kế immutable: field private final, không có setter, defensive copy với mutable object, class có thể final, v.v.

9. final dùng để làm gì?

`final` có 3 nhóm chính.

final variable

```
final int x = 10;
x = 20; // lỗi
```

Biến không được gán lại.

Với object:

```
final List<String> names = new ArrayList<>();  
names.add("A"); // được  
names = new ArrayList<>(); // lỗi
```

final chặn reassign, không chặn mutate.

final method

```
class Parent {  
    final void run() {  
        System.out.println("Parent run");  
    }  
}  
  
class Child extends Parent {  
    void run() { // lỗi  
        System.out.println("Child run");  
    }  
}
```

Method final không được override.

Dùng khi logic đó là logic cốt lõi, không muốn class con thay đổi.

Ví dụ trong hệ thống banking, class cha có method validate transaction flow:

```
public final void process() {  
    validate();  
    debit();  
    credit();  
    audit();  
}
```

Nếu cho override lung tung, class con có thể bỏ qua bước audit hoặc validate, gây lỗi nghiêm trọng.

final class

```
final class SecurityToken {  
}
```

Class `final` không được kế thừa.

Ví dụ nổi tiếng:

```
public final class String
```

`String` là `final` vì nếu cho kế thừa, người khác có thể tạo class con phá tính immutable hoặc phá logic bảo mật.

Ví dụ nếu `String` không `final`, có thể có class con nhìn như `String` nhưng behavior khác đi. Điều này rất nguy hiểm vì `String` được dùng khắp nơi: username, password, file path, class loading, security check, map key, v.v.

10. Vì sao phải tạo private constructor?

Private constructor thường dùng trong vài trường hợp.

Trường hợp 1: Utility class

Ví dụ:

```
</> Java  
  
public final class DateUtils {  
    private DateUtils() {  
        throw new UnsupportedOperationException("Utility class");  
    }  
  
    public static String formatDate(LocalDate date) {  
        return date.toString();  
    }  
}
```

Class này chỉ chứa static method, không cần tạo object:

```
</> Java  
  
DateUtils.formatDate(date);
```

Nếu không private constructor, người khác có thể viết:

```
</> Java  
  
new DateUtils();
```

Điều đó vô nghĩa.

Vì vậy private constructor dùng để chặn việc tạo object.

Trường hợp 2: Singleton

```
</> Java  
  
public class AppConfig {  
    private static final AppConfig INSTANCE = new AppConfig();  
  
    private AppConfig() {  
    }  
  
    public static AppConfig getInstance() {  
        return INSTANCE;  
    }  
}
```

Private constructor đảm bảo bên ngoài không thể tự tạo nhiều object.

Tuy nhiên trong Spring Boot thực tế, mình ít tự viết singleton kiểu này vì Spring container đã quản lý bean singleton cho mình.

Trường hợp 3: Factory method

```
</> Java  
  
public class Money {  
    private final BigDecimal amount;  
  
    private Money(BigDecimal amount) {  
        this.amount = amount;  
    }  
  
    public static Money of(String value) {  
        return new Money(new BigDecimal(value));  
    }  
}
```

Không cho người ngoài gọi constructor trực tiếp. Bắt buộc họ tạo object qua method `of()`, để mình kiểm soát validate, format, default value.

Ví dụ:

```
</> Java
Money.of("100.50");
```

Tốt hơn:

```
</> Java
new Money(...)
```

vì factory method có thể đặt tên rõ nghĩa và che giấu logic tạo object.

11. Vì sao phải dùng final class?

Dùng `final class` khi mình muốn class đó **không bị thay đổi behavior** thông qua kế thừa.

Ví dụ:

```
</> Java
public final class JwtToken {
    private final String value;

    public JwtToken(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }
}
```

Nếu class này không `final`, người khác có thể `extend`:

```
</> Java
class FakeJwtToken extends JwtToken {
    @Override
    public String getValue() {
        return "fake";
    }
}
```

```
}  
}
```

Nếu hệ thống dựa vào object này để xác thực, có thể sinh lỗi hoặc lỗi hổng.

`final` class thường dùng khi:

Một là class immutable, ví dụ `String`, `Integer`, `LocalDate`.

Hai là class liên quan security.

Ba là class có logic business không muốn bị sửa qua inheritance.

Bốn là muốn tránh thiết kế kế thừa sai. Không phải class nào cũng nên cho extend. Nếu chưa thiết kế class để được kế thừa, đôi khi tốt nhất là đóng lại.

Một câu rất thực tế trong Java là:

Design for inheritance, or prohibit it.

Nghĩa là nếu class không được thiết kế cẩn thận để cho người khác kế thừa, thì nên chặn kế thừa bằng `final`.

12. this là gì?

`this` là reference trỏ tới object hiện tại.

Ví dụ:

```
<> Java   
  
class User {  
    private String name;  
  
    User(String name) {  
        this.name = name;  
    }  
}
```

Ở đây:

```
<> Java   
  
this.name
```

là field của object hiện tại.

Còn:

```
</> Java  
  
name
```

là parameter truyền vào constructor.

Nếu không dùng `this` :

```
</> Java  
  
User(String name) {  
    name = name;  
}
```

Dòng này vô nghĩa, vì parameter tự gán cho chính nó. Field không được set.

13. Các cách dùng phổ biến của `this`

Phân biệt field và parameter

```
</> Java  
  
this.name = name;
```

Đây là cách dùng nhiều nhất.

Gọi constructor khác trong cùng class

```
</> Java  
  
class User {  
    String name;  
    int age;  
  
    User(String name) {  
        this(name, 18);  
    }  
  
    User(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
}  
}
```

Lưu ý:

```
</> Java  
  
this(...)
```



phải là dòng đầu tiên trong constructor.

Trả về chính object hiện tại

Hay dùng trong builder hoặc fluent API:

```
</> Java  
  
class UserBuilder {  
    private String name;  
  
    UserBuilder name(String name) {  
        this.name = name;  
        return this;  
    }  
}
```



Dùng:

```
</> Java  
  
new UserBuilder()  
    .name("Eric");
```



Không dùng được trong static context

```
</> Java  
  
class User {  
    static void test() {  
        System.out.println(this); // lỗi  
    }  
}
```



Vì static method thuộc về class, không thuộc về object cụ thể nào. Không có object hiện tại thì không có `this`.

14. super là gì?

`super` dùng để tham chiếu tới phần cha của object hiện tại.

Ví dụ:

```
</> Java

class Animal {
    void speak() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void speak() {
        super.speak();
        System.out.println("Dog barks");
    }
}
```

Khi gọi:

```
</> Java

new Dog().speak();
```

Output:

```
</> Java

Animal sound
Dog barks
```

`super.speak()` gọi method của class cha.

15. Các cách dùng phổ biến của `super`

Gọi constructor cha

</> Java



```
class Animal {
    Animal(String name) {
        System.out.println(name);
    }
}

class Dog extends Animal {
    Dog() {
        super("Dog");
    }
}
```

`super(...)` phải là dòng đầu tiên trong constructor.

Nếu class cha không có constructor rỗng, class con bắt buộc phải gọi đúng constructor cha.

Gọi method cha khi bị override

</> Java



```
@Override
void process() {
    super.process();
    // thêm logic riêng
}
```

Hay dùng khi muốn giữ logic gốc rồi mở rộng thêm.

Truy cập field cha bị che khuất

</> Java



```
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;

    void print() {
        System.out.println(this.x); // 20
        System.out.println(super.x); // 10
    }
}
```

```
}  
}
```

Nhưng thực tế nên tránh đặt field trùng tên như vậy vì code dễ rối.

16. Vì sao cần abstract class và interface?

Đây là phần cực kỳ quan trọng.

Trước tiên phải hiểu vấn đề gốc: trong OOP, mình không chỉ viết class để chạy, mà còn viết class để **thiết kế quan hệ giữa các thành phần**.

Nếu chỉ có class bình thường, mình sẽ gặp hai vấn đề lớn:

Một là không biểu diễn được "khái niệm chung chưa hoàn chỉnh".

Hai là khó tách hệ thống khỏi implementation cụ thể.

Ví dụ trong hệ thống payment:

</> Java

```
class MomoPayment {  
    void pay() {  
        // gọi API Momo  
    }  
}  
  
class StripePayment {  
    void pay() {  
        // gọi API Stripe  
    }  
}
```

Nếu service viết trực tiếp như này:

</> Java

```
class OrderService {  
    private MomoPayment payment = new MomoPayment();  
  
    void checkout() {  
        payment.pay();  
    }  
}
```

Thì OrderService bị dính cứng vào MomoPayment .

Mai muốn đổi sang Stripe, test mock payment, thêm PayPal, thêm bank transfer — code sẽ khó mở rộng.

Vì vậy mình tạo interface:

</> Java



```
interface PaymentGateway {  
    void pay();  
}
```

Rồi implementation:

</> Java



```
class MomoPayment implements PaymentGateway {  
    public void pay() {  
        System.out.println("Pay by Momo");  
    }  
}  
  
class StripePayment implements PaymentGateway {  
    public void pay() {  
        System.out.println("Pay by Stripe");  
    }  
}
```

Service chỉ phụ thuộc vào abstraction:

</> Java



```
class OrderService {  
    private final PaymentGateway paymentGateway;  
  
    OrderService(PaymentGateway paymentGateway) {  
        this.paymentGateway = paymentGateway;  
    }  
  
    void checkout() {  
        paymentGateway.pay();  
    }  
}
```

Bây giờ OrderService không cần biết bên dưới là Momo, Stripe, PayPal hay mock object.

Đây là lý do interface sinh ra: **định nghĩa contract để các class khác tuân theo, giúp hệ thống mở rộng mà ít sửa code cũ.**

17. Interface là gì?

Interface là một **hợp đồng hành vi**.

Nó nói rằng:

Ai implement interface này thì bắt buộc phải có những method này.

Ví dụ:

  NMH  

```
interface Flyable {  
    void fly();  
}
```

Class nào implement `Flyable` thì phải biết bay:

 Java 

```
class Bird implements Flyable {  
    public void fly() {  
        System.out.println("Bird flies");  
    }  
}  
  
class Airplane implements Flyable {  
    public void fly() {  
        System.out.println("Airplane flies");  
    }  
}
```

Điểm hay là `Bird` và `Airplane` không cùng họ hàng kế thừa. Một cái là động vật, một cái là máy móc. Nhưng cả hai có cùng hành vi `fly()`.

Đây là thứ class inheritance không làm tốt.

Nếu dùng class cha `FlyingThing`, sẽ rất gượng ép vì chim và máy bay không thật sự có cùng bản chất. Nhưng chúng có cùng capability: biết bay.

Vì vậy interface rất phù hợp để mô tả năng lực:

 Java 

```
Runnable  
Comparable  
Serializable
```

Closeable
PaymentGateway
NotificationSender
UserRepository

18. Abstract class là gì?

Abstract class là class chưa hoàn chỉnh. Nó có thể có:

- Field
- Constructor
- Method thường đã có code
- Abstract method chưa có code
- Protected method
- State dùng chung
- Template method

Ví dụ:

</> Java

```
abstract class BaseReportExporter {  
    public final void export() {  
        loadData();  
        formatData();  
        writeFile();  
    }  
  
    protected void loadData() {  
        System.out.println("Load data from database");  
    }  
  
    protected abstract void formatData();  
  
    protected abstract void writeFile();  
}
```

Class con:

</> Java

```
class PdfReportExporter extends BaseReportExporter {  
    protected void formatData() {  
        System.out.println("Format PDF");  
    }  
  
    protected void writeFile() {
```

```
        System.out.println("Write PDF file");
    }
}

class ExcelReportExporter extends BaseReportExporter {
    protected void formatData() {
        System.out.println("Format Excel");
    }

    protected void writeFile() {
        System.out.println("Write Excel file");
    }
}
```

Ở đây abstract class giữ phần flow chung:

</> Java



```
loadData();
formatData();
writeFile();
```

Nhưng cho class con tự quyết định format và write file.

Đây là kiểu thiết kế rất thực tế, gọi là **Template Method Pattern**.

19. Vì sao không dùng class bình thường luôn?

Vì class bình thường biểu diễn một thứ có thể tạo object hoàn chỉnh.

Ví dụ:

</> Java



```
class Animal {
}
```

Bạn có thể viết:

</> Java



```
new Animal();
```

Nhưng đôi khi `Animal` là khái niệm quá chung. Trong thực tế, không có con vật cụ thể tên là "Animal". Có `Dog`, `Cat`, `Bird`.

Vì vậy:

```
</> Java

abstract class Animal {
    abstract void makeSound();
}
```

Không cho tạo:

```
</> Java

new Animal(); // lỗi
```

Mình bắt buộc tạo class cụ thể:

```
</> Java

class Dog extends Animal {
    void makeSound() {
        System.out.println("Woof");
    }
}
```

Abstract class dùng khi class cha có ý nghĩa chung nhưng chưa đủ cụ thể để tạo object.

20. Interface vs abstract class khác nhau ở đâu?

Nói ngắn gọn:

Interface tập trung vào "class này làm được gì".

Abstract class tập trung vào "class này là một loại gì, và có sẵn một phần logic chung".

Ví dụ:

```
</> Java

interface Payable {
    void pay();
}
```

Cái gì trả tiền được thì implement Payable .

Còn:

</> Java



```
abstract class Employee {  
    protected String name;  
    protected BigDecimal salary;  
  
    abstract BigDecimal calculateSalary();  
}
```

Employee là một loại người trong domain công ty. Nó có state chung như name, salary.

21. Khi nào dùng interface?

Dùng interface khi muốn định nghĩa contract và cho nhiều class khác nhau implement.

Ví dụ trong Spring Boot:

</> Java



```
public interface UserRepository {  
    User findById(Long id);  
}
```

Service phụ thuộc vào interface:

</> Java



```
public class UserService {  
    private final UserRepository userRepository;  
  
    public UserService(UserRepository userRepository) {  
        this.userRepository = userRepository;  
    }  
}
```

Ưu điểm là sau này có thể thay implementation:

</> Java



```
class JpaUserRepository implements UserRepository  
class RedisUserRepository implements UserRepository  
class MockUserRepository implements UserRepository
```

Code service không cần sửa.

Dùng interface khi:

- Muốn loose coupling.
- Muốn dễ test mock.
- Muốn nhiều implementation.
- Muốn định nghĩa capability.
- Muốn tuân theo Dependency Inversion Principle.
- Không cần chia sẻ state chung.
- Không muốn ép class vào một cây kế thừa cứng.

Ví dụ thực tế:

Java

```
interface EmailSender {  
    void send(String to, String content);  
}  
  
class SmtplibEmailSender implements EmailSender {  
    public void send(String to, String content) {  
        // send SMTP  
    }  
}  
  
class AwsSesEmailSender implements EmailSender {  
    public void send(String to, String content) {  
        // send AWS SES  
    }  
}
```

Service chỉ biết:

Java

```
private final EmailSender emailSender;
```

Không quan tâm bên dưới gửi bằng SMTP hay AWS SES.

22. Khi nào dùng abstract class?

Dùng abstract class khi các class con có cùng bản chất và có nhiều logic/state dùng chung.

Ví dụ:

Java

```
abstract class BaseEntity {  
    protected Long id;  
    protected LocalDateTime createdAt;  
    protected LocalDateTime updatedAt;  
}
```

Hoặc:

</> Java



```
abstract class AbstractPaymentProcessor {  
    public final void processPayment() {  
        validate();  
        executePayment();  
        saveTransaction();  
        notifyUser();  
    }  
  
    protected void validate() {  
        System.out.println("Common validation");  
    }  
  
    protected abstract void executePayment();  
  
    protected void saveTransaction() {  
        System.out.println("Save transaction");  
    }  
  
    protected void notifyUser() {  
        System.out.println("Notify user");  
    }  
}
```

Class con chỉ custom phần khác nhau:

</> Java



```
class CreditCardProcessor extends AbstractPaymentProcessor {  
    protected void executePayment() {  
        System.out.println("Pay by credit card");  
    }  
}  
  
class BankTransferProcessor extends AbstractPaymentProcessor {  
    protected void executePayment() {  
        System.out.println("Pay by bank transfer");  
    }  
}
```

Dùng abstract class khi:

- Có logic chung muốn reuse.
- Có field chung.
- Có constructor chung.
- Muốn kiểm soát flow bằng template method.
- Các class con thật sự là cùng một họ.
- Muốn cung cấp default behavior mạnh hơn interface.

23. Nên dùng interface hay abstract class?

Trong thiết kế hiện đại, thường ưu tiên:

</> Java



```
interface trước, abstract class sau
```

Nhưng không có nghĩa interface luôn tốt hơn.

Dùng interface khi bạn chỉ cần contract:

</> Java



```
interface PaymentGateway {  
    void pay();  
}
```

Dùng abstract class khi bạn cần chia sẻ logic:

</> Java



```
abstract class AbstractPaymentGateway implements PaymentGateway {  
    protected void validateAmount(BigDecimal amount) {  
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {  
            throw new IllegalArgumentException("Invalid amount");  
        }  
    }  
}
```

Một cách kết hợp rất hay trong thực tế:

</> Java



```
interface PaymentGateway {  
    void pay(BigDecimal amount);  
}  
  
abstract class AbstractPaymentGateway implements PaymentGateway {
```

```
protected void validate(BigDecimal amount) {
    if (amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("Amount must be positive");
    }
}

class StripePaymentGateway extends AbstractPaymentGateway {
    public void pay(BigDecimal amount) {
        validate(amount);
        // Stripe logic
    }
}

class PaypalPaymentGateway extends AbstractPaymentGateway {
    public void pay(BigDecimal amount) {
        validate(amount);
        // Paypal logic
    }
}
```

Interface là contract bên ngoài nhìn vào.

Abstract class là base hỗ trợ reuse nội bộ.

Đây là cách thiết kế khá sạch.

24. Vì sao Java không cho phép đa kế thừa class?

Java không cho phép:

```
</> Java

class C extends A, B {
}
```



Lý do lớn nhất là tránh ambiguity, đặc biệt là **diamond problem**.

Ví dụ giả sử Java cho phép:

```
</> Java

class A {
    void hello() {
        System.out.println("A");
    }
}
```



```
class B extends A {  
    void hello() {  
        System.out.println("B");  
    }  
}  
  
class C extends A {  
    void hello() {  
        System.out.println("C");  
    }  
}  
  
class D extends B, C {  
}
```

Bây giờ gọi:

```
</> Java  
  
new D().hello();
```



Nên in ra gì?

```
</> Java  
  
B
```



hay

```
</> Java  
  
C
```



Không rõ.

Đó là diamond problem.

Java chọn thiết kế đơn giản hơn:

```
</> Java  
  
class Dog extends Animal
```



Một class chỉ extend một class cha.

Nhưng Java vẫn cho phép implement nhiều interface:

```
</> Java
```



```
class Duck implements Flyable, Swimmable, Walkable {  
}
```

Vì interface ban đầu chỉ định nghĩa contract, không có state implementation phức tạp như class.

25. Nhưng interface có default method thì có bị diamond problem không?

Từ Java 8, interface có default method .

Ví dụ:

```
</> Java  
  
interface A {  
    default void hello() {  
        System.out.println("A");  
    }  
}  
  
interface B {  
    default void hello() {  
        System.out.println("B");  
    }  
}  
  
class C implements A, B {  
}
```

Code này lỗi.

Java bắt class C tự override:

```
</> Java  
  
class C implements A, B {  
    public void hello() {  
        A.super.hello();  
    }  
}
```

Hoặc:

```
</> Java
```

```
class C implements A, B {  
    public void hello() {  
        System.out.println("C");  
    }  
}
```

Java không tự đoán. Nó bắt developer quyết định rõ ràng.

Đây là cách Java cho phép nhiều interface nhưng vẫn kiểm soát xung đột.

26. Class thắng interface

Một câu bẫy nữa:

```
</> Java  
  
class Parent {  
    public void hello() {  
        System.out.println("Parent");  
    }  
}  
  
interface A {  
    default void hello() {  
        System.out.println("A");  
    }  
}  
  
class Child extends Parent implements A {  
}
```

Gọi:

```
</> Java  
  
new Child().hello();
```

Output:

```
</> Java  
  
Parent
```

Quy tắc là:

```
</> Java
```



```
class wins over interface
```

Nếu class cha đã có method cụ thể, default method trong interface không được ưu tiên.

27. Abstract class có constructor không?

Có.

```
</> Java

abstract class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
}

class Dog extends Animal {
    Dog() {
        System.out.println("Dog constructor");
    }
}
```

Gọi:

```
</> Java

new Dog();
```

Output:

```
</> Java

Animal constructor
Dog constructor
```

Dù abstract class không tạo object trực tiếp được, constructor của nó vẫn chạy khi class con được tạo.

Vì object Dog vẫn có phần Animal bên trong.

28. Interface có constructor không?

Không.

Interface không đại diện cho object cụ thể và không giữ object state theo kiểu class, nên không có constructor.

Interface có thể có constant:

```
</> Java

interface A {
    int X = 10;
}
```

Thực chất field trong interface mặc định là:

```
</> Java

public static final
```

Nên:

```
</> Java

A.X
```

là hằng số thuộc interface, không phải state riêng của từng object.

29. Interface có field không?

Có, nhưng field trong interface luôn là:

```
</> Java

public static final
```

Ví dụ:

```
</> Java

interface Config {
    int TIMEOUT = 30;
}
```

Tương đương:

```
</> Java
```

```
public static final int TIMEOUT = 30;
```

Không nên dùng interface để chứa quá nhiều constant business linh tinh. Ngày xưa có pattern gọi là constant interface, nhưng hiện nay thường không khuyến khích vì class implement interface chỉ để lấy constant là sai mục đích.

30. Abstract class có field không?

Có.

</> Java

```
abstract class BaseService {  
    protected Logger logger;  
}
```



Abstract class có thể giữ state dùng chung cho class con.

Đây là một khác biệt lớn với interface.

31. Abstract method là gì?

Abstract method là method chỉ khai báo, chưa có body.

</> Java

```
abstract class Animal {  
    abstract void speak();  
}
```



Class con bắt buộc implement:

</> Java

```
class Dog extends Animal {  
    void speak() {  
        System.out.println("Woof");  
    }  
}
```



Nếu class con không implement hết abstract method, class con cũng phải là abstract.

32. Interface có abstract method không?

Có. Trước Java 8, interface chủ yếu chứa abstract method.

Ví dụ:

</> Java

```
interface Runnable {  
    void run();  
}
```



Tương đương:

</> Java

```
public abstract void run();
```



Method trong interface mặc định là `public abstract`, nếu không phải `default`, `static`, hoặc `private`.

33. Interface có private method không?

Từ Java 9, interface có thể có private method để tái sử dụng logic bên trong các default method.

Ví dụ:

</> Java

```
interface Logger {  
    default void info(String msg) {  
        log("INFO", msg);  
    }  
  
    default void error(String msg) {  
        log("ERROR", msg);  
    }  
  
    private void log(String level, String msg) {  
        System.out.println(level + ": " + msg);  
    }  
}
```



Private method trong interface chỉ phục vụ nội bộ interface, class implement không gọi trực tiếp được.

34. Một class có thể vừa extend abstract class vừa implement interface không?

Có.

</> Java



```
interface Auditable {  
    void audit();  
}  
  
abstract class BaseService {  
    protected void validate() {  
        System.out.println("validate");  
    }  
}  
  
class OrderService extends BaseService implements Auditable {  
    public void audit() {  
        System.out.println("audit");  
    }  
}
```

Java chỉ cấm extend nhiều class, không cấm implement nhiều interface.

35. Interface có thay thế abstract class hoàn toàn không?

Không.

Interface mạnh hơn nhiều từ Java 8 vì có default method, static method, private method. Nhưng interface vẫn không thay thế hoàn toàn abstract class.

Interface không phù hợp khi cần:

- State instance dùng chung.
- Constructor.
- Protected field.
- Non-public abstract method.
- Template method phức tạp.
- Quan hệ "is-a" mạnh giữa các class con.

Ví dụ `AbstractList`, `AbstractMap`, `AbstractCollection` trong Java Collections tồn tại vì có rất nhiều logic chung giữa các collection. Nếu chỉ dùng interface, mỗi

implementation phải viết lại quá nhiều code.

36. Abstract class có thay thế interface hoàn toàn không?

Cũng không.

Vì Java chỉ cho extend một class. Nếu dùng abstract class làm contract, class con mất quyền extend class khác.

Ví dụ:

</> Java

```
abstract class PaymentGateway {  
    abstract void pay();  
}
```



Nếu class `StripePaymentGateway` đã cần extend một base class khác thì không extend được `PaymentGateway`.

Trong khi với interface:

</> Java

```
class StripePaymentGateway extends SomeBaseClass implements PaymentGateway {  
}
```



Vẫn được.

Vì vậy interface linh hoạt hơn nhiều để định nghĩa contract.

37. Câu bầy: abstract class có method thường được không?

Có.

</> Java

```
abstract class Animal {  
    void breathe() {  
        System.out.println("Breathing");  
    }  
}
```



```
abstract void speak();  
}
```

Abstract class không bắt buộc tất cả method phải abstract.

Một abstract class thậm chí có thể không có abstract method nào:

```
</> Java  
  
abstract class BaseEntity {  
    Long id;  
}
```

Nó vẫn abstract nếu mình không muốn ai tạo object trực tiếp từ nó.

38. Câu bẫy: class thường có abstract method được không?

Không.

```
</> Java  
  
class Animal {  
    abstract void speak(); // lỗi  
}
```

Nếu class có abstract method, class đó bắt buộc phải khai báo `abstract` .

39. Câu bẫy: interface method có cần public không?

Trong interface:

```
</> Java  
  
interface A {  
    void run();  
}
```

`run()` mặc định là:

```
</> Java  
  
public abstract void run();
```

Khi implement, không được giảm visibility:

```
</> Java

class B implements A {
    void run() { // lỗi
    }
}
```

Phải là:

```
</> Java

class B implements A {
    public void run() {
    }
}
```

Vì method trong interface là public, class implement không được biến nó thành package-private.

40. Câu bẫy: abstract class có final abstract method được không?

Không.

```
</> Java

abstract class A {
    final abstract void run(); // lỗi
}
```

Vì abstract nghĩa là bắt class con override.

final nghĩa là cấm override.

Hai cái mâu thuẫn nhau.

41. Câu bẫy: final class có abstract được không?

Không.

```
</> Java
```



```
final abstract class A {  
}
```

abstract class cần class con kế thừa để hoàn thiện.

final class cấm kế thừa.

Mâu thuẫn.

42. Câu bẫy: private method có override được không?

Không.

```
</> Java  
  
class Parent {  
    private void run() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
    private void run() {  
        System.out.println("Child");  
    }  
}
```

Đây không phải override. Private method không được kế thừa theo nghĩa polymorphism. Method run() trong Child là method riêng khác.

43. Câu bẫy: static method có override không?

Không, static method bị hidden, không override.

```
</> Java  
  
class Parent {  
    static void hello() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
    static void hello() {  
        System.out.println("Child");  
    }  
}
```

```
}

public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.hello();
    }
}
```

Output:

</> Java



Parent

Vì static method được resolve theo type của biến reference lúc compile-time, không theo object runtime.

Còn instance method thì polymorphism:

</> Java



```
class Parent {
    void hello() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    void hello() {
        System.out.println("Child");
    }
}

Parent p = new Child();
p.hello(); // Child
```

44. Câu bẫy với field: field không polymorphic

</> Java



```
class Parent {
    String name = "Parent";
}

class Child extends Parent {
    String name = "Child";
}
```

```
}

public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        System.out.println(p.name);
    }
}
```

Output:

</> Java



Parent

Field không override theo kiểu method. Field bị hidden và được resolve theo type của biến reference.

Nhưng nếu là method:

</> Java



```
class Parent {
    String getName() {
        return "Parent";
    }
}

class Child extends Parent {
    String getName() {
        return "Child";
    }
}

Parent p = new Child();
System.out.println(p.getName()); // Child
```

Method instance có polymorphism.

45. Câu bẫy Integer cache

</> Java



```
Integer a = 127;
Integer b = 127;

System.out.println(a == b);
```

Output:

```
</> Java  
  
true
```



Nhưng:

```
</> Java  
  
Integer a = 128;  
Integer b = 128;  
  
System.out.println(a == b);
```



Output thường là:

```
</> Java  
  
false
```



Vì Java cache wrapper Integer trong khoảng -128 đến 127 . Với số trong khoảng này, autoboxing có thể dùng lại object cũ.

Nhưng ngoài khoảng đó, có thể tạo object khác.

Không nên so sánh wrapper bằng `==`.

Nên dùng:

```
</> Java  
  
a.equals(b)
```



Hoặc nếu cần so sánh số:

```
</> Java  
  
a.intValue() == b.intValue()
```



Nhưng nhớ check null nếu wrapper có thể null.

46. Câu bẫy autoboxing với null

</> Java



```
Integer x = null;  
int y = x;
```

Lỗi runtime:

</> Java



```
NullPointerException
```

Vì Java cố unbox Integer thành int , tương đương:

</> Java



```
x.intValue()
```

Nhưng x đang null.

Trong dự án thật, lỗi này rất hay gặp khi field database kiểu nullable map vào Integer , Long , Boolean , rồi code tự động unbox.

Ví dụ nguy hiểm:

</> Java



```
Boolean active = user.getActive();  
  
if (active) {  
    // NPE nếu active null  
}
```

Nên viết an toàn:

</> Java



```
if (Boolean.TRUE.equals(active)) {  
}
```

47. Câu bẫy equals() và hashCode()

Nếu override equals() mà không override hashCode() , các collection như HashMap , HashSet có thể hoạt động sai.

Ví dụ:

</> Java



```
class User {
    Long id;

    User(Long id) {
        this.id = id;
    }

    public boolean equals(Object o) {
        if (!(o instanceof User)) return false;
        return Objects.equals(id, ((User) o).id);
    }
}
```

Dùng:

</> Java



```
Set<User> users = new HashSet<>();
users.add(new User(1L));
users.add(new User(1L));

System.out.println(users.size());
```

Bạn mong là 1 , nhưng có thể ra 2 vì hashCode() không đồng bộ với equals() .

Contract là:

Nếu hai object bằng nhau theo equals() , thì hashCode() phải bằng nhau.

Code đúng:

</> Java



```
class User {
    Long id;

    User(Long id) {
        this.id = id;
    }

    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof User)) return false;
        User user = (User) o;
        return Objects.equals(id, user.id);
    }

    public int hashCode() {
```

```
        return Objects.hash(id);
    }
}
```

48. Tóm lại về interface và abstract class trong thiết kế thực tế

Nếu đang thiết kế service trong Spring Boot, thường nên bắt đầu bằng interface:

```
</> Java

public interface PaymentService {
    void pay(PaymentRequest request);
}
```

Sau đó có implementation:

```
</> Java

@Service
public class StripePaymentService implements PaymentService {
    public void pay(PaymentRequest request) {
        // logic
    }
}
```

Nếu có nhiều implementation dùng chung validate, logging, template flow, có thể thêm abstract class:

```
</> Java

public abstract class AbstractPaymentService implements PaymentService {
    protected void validate(PaymentRequest request) {
        // common validation
    }
}
```

Rồi:

```
</> Java

@Service
public class StripePaymentService extends AbstractPaymentService {
    public void pay(PaymentRequest request) {
        validate(request);
        // Stripe logic
    }
}
```

```
}  
}
```

Cách hiểu tốt nhất:

Java



```
interface = hợp đồng bên ngoài  
abstract class = khung xử lý / logic dùng chung bên trong  
class = implementation cụ thể chạy thật
```

49. Một số keyword cần nhớ khi đi phỏng vấn

Pass by value: Java luôn truyền bản sao giá trị. Với object, giá trị đó là reference.

Reference: Biến object không chứa object trực tiếp, mà chứa tham chiếu tới object.

Mutable: Object có thể thay đổi state sau khi tạo, ví dụ `StringBuilder` , `ArrayList` .

Immutable: Object không thay đổi state sau khi tạo, ví dụ `String` , `Integer` , `LocalDate` .

String Pool: Vùng JVM dùng để tái sử dụng String literal.

`==` : So sánh primitive value hoặc so sánh reference với object.

`equals()` : So sánh logic bằng nhau của object, thường là nội dung.

`hashCode()` : Mã hash dùng trong `HashMap` , `HashSet` . Phải nhất quán với `equals()` .

`final variable` : Không được gán lại biến.

`final method` : Không được override.

`final class` : Không được kế thừa.

`this` : Object hiện tại.

`super` : Phần cha của object hiện tại.

Abstract class: Class chưa hoàn chỉnh, dùng khi cần logic/state chung.

Interface: Contract hành vi, dùng để loose coupling và polymorphism.

Diamond problem: Vấn đề mơ hồ khi một class kế thừa nhiều nhánh có cùng method.

Default method: Method có body trong interface, xuất hiện từ Java 8.

Polymorphism: Đa hình. Biến kiểu cha/interface có thể trở tới object class con, và method runtime của class con được gọi.

Autoboxing/unboxing: Tự động chuyển giữa primitive và wrapper, ví dụ `int` ↔ `Integer` .

Static method hiding: Static method không override, chỉ bị che khuất theo kiểu biến reference.

